IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 8 September 2025, accepted 23 September 2025,
date of publication 25 September 2025, date of current version 2 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3614553

APPLIED RESEARCH

Context-Aware Retrieval Augmented Generation Using Similarity Validation to Handle Context Inconsistencies in Large Language Models

ENRICO COLLINI, FELIX INDRA KURNIADI, PAOLO NESI, (Life Member, IEEE), AND GIANNI PANTALEO

Distributed Systems and Internet Technologies Laboratory (DISIT Lab), Department of Information Engineering, University of Florence, 50139 Florence, Italy

Corresponding author: Paolo Nesi (paolo.nesi@unifi.it)

This work was supported by the Ministry University of Research (MUR), project CAI4DSA of FAIR PE1.

ABSTRACT Large Language Models (LLMs) have transformed natural language processing by offering human-like responses. However, issues such as incorrect information (hallucinations) and errors in specific subject areas remain, especially in Retrieval Augmented Generation (RAG) systems. This study introduces a Context-Aware Retrieval Augmented Generation (CA-RAG), which simplifies the process by removing the need to separately find relevant chunks of a document. Instead, after dividing the document into chunks, both the question and chunks are directly given to the LLMs to produce answers. The method then focuses on improving the answers through additional post-processing, aiming to reduce errors and make the answers more relevant to the question. To evaluate the effectiveness of CA-RAG, two scenarios have been designed. Scenario 1 involved experiments using widely adopted and recognized benchmark datasets, such as TriviaQA, Natural Questions, AmbigQA and Stanford Question Answering Dataset (SQuAD). In this context, the proposed CA-RAG method, combined with similarity measure (either cosine similarity or dot product) between generated answers and chunks, achieved the highest F1-score in TriviaQA and AmbigQA. Scenario 2 tested CA-RAG robustness using a custom dataset comprising domain-specific and unstructured documents. Results from automated and manual evaluations revealed that CA-RAG with post-processing consistently outperformed traditional RAG. These findings highlight the critical role of post-processing techniques and similarity measures in improving the accuracy and relevance of generated answers. CA-RAG demonstrates strong potential as a reliable and versatile solution for Retrieval Augmented Generation tasks across diverse datasets and domains.

INDEX TERMS Context-aware retrieval augmented generation (CA-RAG), cosine similarity, dot product, domain-specific tasks, large language models (LLMs), retrieval-augmented generation (RAG).

I. INTRODUCTION

Large Language Models (LLMs) are one of the most important developments in Artificial Intelligence. LLMs like ChatGPT [1], Claude [2], and Llama [3] can create text with strong "understanding" and answers to questions in ways that make sense to humans. These models use deep neural networks trained on large amounts of text data, which allows them to understand and produce human-like language.

Many industries, including healthcare [4], [5], education [6], [7], [8], and finance [9], [10], [11], now use LLMs to support decision-making. However, while LLMs are powerful tools, they still have some weaknesses, including potential biases inherent in their training data, generation of incorrect or inaccurate information (known as hallucinations), as well as high development and operational costs. These problems are particularly evident when LLMs handle queries outside their pretrained data or require domain-specific knowledge [12], [13].

Moreover, training and fine-tuning LLMs on specific domain knowledge, which may contain sensitive and/or private information, can expose and amplify vulnerabilities to which such systems are prone, for instance prompt injection attacks and several other adversarial techniques, which may

The associate editor coordinating the review of this manuscript and approving it for publication was Ayman El-Baz.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

lead to training data exposure, and therefore significant privacy risks [14]. On the other hand, fine-tuning LLMs locally on large-scale data requires high costs in terms of computational resources [15], since fine-tuning strongly relies on the availability of vast amounts of training data, and this aspect is often limited in specialized domains. For these reasons, fine-tuning may not be the best approach in all those cases and contexts in which users, research entities, organizations and companies require to build LLM-based applications and services for specific domain knowledge (products and/or services manuals, internal/legal documentation etc.) which may need frequent or continuous updates.

To address these challenges, the Retrieval-Augmented Generation (RAG) approach [16] has been considered. This method improves LLM performance by retrieving relevant information from pre-existing documents and using it to generate answers. In this way, the RAG approach reduces the need for exhaustive fine-tuning by leveraging up-to-date information and specific domain knowledge, therefore minimizing computational overhead and related costs. RAG combines two processes:

1) Retrieval: The system retrieves relevant information from external documents or sources.
2) Generation: The retrieved information is then used to generate responses based on the query.

By integrating these two processes, RAG is a scalable and efficient solution that helps LLMs produce answers that may result to be more reliable on domain-specific questions. This approach reduces the impact of hallucinations and improves the factual accuracy by incorporating verified external knowledge.

In general, Lewis et al. RAG [16] works using the "Retrieve and Read" framework. This method improves LLMs by adding external knowledge through steps such as chunking data, storing it in vector format, and retrieving relevant information based on similarity. However, existing methods face several limitations [13], [17]. Actually, several works have shown that proper selection of retrieval methods can significantly enhance system performance [18], [19]. However, an incorrect selection of the retriever can significantly affect the outcomes of the generation process, and may lead to missing key information in specific contexts. For instance, if the answer to a question is spread across several different sections of a document (that may not be the most semantically similar), or even in different documents, a traditional RAG framework might not be able to retrieve the most relevant chunks [20].

A. MOTIVATION AND MAIN OBJECTIVE

The use of RAG may provide several notable advantages with respect to LLM fine tuning in different cases. First, RAG is designed to use context for specific tasks, enabling it to generate more relevant and tailored outputs in Question and Answer tasks on specific domain knowledge. Therefore, RAG allows to handle queries that go beyond the scope of pre-trained models, since it excels at managing specialized domain knowledge retrieved from external sources [21]. Moreover, RAG minimizes the risk of data leakage from the pre-trained model, making it a reliable option for maintaining privacy [22]. Furthermore, while fine-tuning offers consistent performance, RAG provides a more efficient and cost-effective alternative [23]. However, while efficient, the traditional RAG approach introduces a critical bottleneck: the overall answer quality heavily depends on the retriever's performance, as well as on the precision of the contextual information used to generate answers [24]. Previous research has identified computational complexity [21], [25] and retrieval challenges [26], [27] as significant weaknesses in current RAG. In particular, similarity measures used in the retrieval stage may not effectively capture the contextual relevance between a query and potentially relevant documents [28], [29]. Furthermore, dense vector representations, such as document-level embedding models may not be enough to capture important and semantically significant content [24]. This may cause a semantic drift, where relevant chunks are filtered out early, leading to incomplete or inaccurate answers. Therefore, the traditional RAG approach may often lead to the retrieval of documents or text chunks which may be lexically similar but semantically misaligned with the user's expectations. Therefore, despite its many advantages, traditional RAG has several significant limitations. In order to address all the described issues, this study proposes the Context-Aware Retrieval Augmented Generation (CA-RAG) solution.

B. PROPOSED METHOD AND CONTRIBUTIONS

The CA-RAG approach focuses on improving context awareness and mitigating retrieval precision issues by eliminating the retrieval initial filter. Rather than ranking raw documents based on vector similarity to the query, our proposed method consists in performing semantic chunking on documents and providing all the extracted chunks to the LLM to generate answers. Then, we rank generated answers using similarity methods between the generated text and the context, discarding answers whose similarity score undergoes a certain threshold. This new approach moves the relevance judgment to the LLM's own interpretative ability, which can be exploited on the whole unfiltered documentation. This lays the foundation for obtaining more precise relevance estimation and more complete answers (as outlined by the results illustrated in the experimental section), since the generated responses may reflect a deeper contextual alignment with queries and user expectations. Therefore, even if the proposed approach might be more expensive in terms of response times than traditional RAG approaches, the main focus of our work is to emphasize that there are many real-world scenarios where answer accuracy and completeness are much more important than performance. This aspect is relevant in many specialized contexts such as medical [30] or legal [31] domains, where response

accuracy and mitigation of spreading misinformation have great importance [30], even at the expense of increased computational cost, since users in these scenarios may often prioritize the precision and reliability of answers over minimizing response times.

To summarize, the CA-RAG model introduces a straightforward yet effective method that uses similarity methods evaluation to assess the relevance of responses generated by the LLM to the given context. Unlike traditional RAG methods that rely on top-k retrieval results, CA-RAG directly evaluates the relevance of each retrieved chunk. This reduces dependency on complex ranking systems and improves retrieval accuracy.

The following research objectives guide this study:

1) To propose the CA-RAG method and evaluate its effectiveness without relying on the top-k retrieval approach.
2) To implement Semantic Chunking and Fixed-Size Chunking in the document chunking process, ensuring that contextual information is preserved.
3) To propose the use of similarity methods evaluation during the generation phase to improve the alignment between retrieval and response generation.

The paper is organized as follows: Section II provides a review of state of the art RAG approaches, techniques and implementations found in literature, focusing also on comparative performance analysis. Section III introduces the proposed CA-RAG model, detailing its components and processes. Section IV presents the experiments, results, and analysis of the proposed model. Finally, Section V is left for conclusions, outlining also potential directions for future research.

II. RELATED WORKS

Several studies suggest addressing RAG using different approaches. Table 1 provides a comparative overview of the current best practices in RAG that have been reviewed in the literature.

Pickett et al. [25] suggested a novel paradigm named Dartboard. This study introduces the Dartboard algorithm, which uses a simple measure of total information gain for a specific query, and evaluates the effectiveness of Dartboard on the RAG Benchmark. Their system was compared with other methods including Oracle, Maximal Marginal Relevance (MMR) Crosscoder, MMR Cossim, K-nearest-neighbors (KNN) Crosscoder, KNN Cossim. The highest Question Answer (QA) score for Dartboard (see Table 1) is Exact String Match score of 85.6% (obtained performing a string match of the response on a set of correct answers, thus making each response as either correct or incorrect). Then, the highest Normalized Discounted Cumulative Gain (NDCG) score is 0.975. An inherent drawback of Dartboard is its computational complexity and hyperparameter sensitivity.

Yan et.al. [26] introduced a technique called Corrective RAG (CRAG) aimed at improving robustness, automatic self-correction, and the effective exploitation of retrieved documents in RAG systems. CRAG begins with an initial retrieval phase in which the user's query is sent to a retriever to obtain the top-k documents. Each resulting question-answer pair is then evaluated by a T5-large-based retrieval evaluator, which assigns a relevance score and classifies the pair into three categories: correct, incorrect, or ambiguous. The corrective procedure unfolds as follows: if the evaluator labels a pair as correct, the corresponding document is split into small fragments (''knowledge strips''); each strip is re-evaluated and, if deemed irrelevant, discarded, whereas relevant strips are merged into the internal knowledge store. If a pair is judged as incorrect, all retrieved documents are considered useless, the question is rewritten, and a web search is performed; the newly retrieved information is filtered in the same way as in the correct phase and stored as external knowledge. If the evaluator deems a pair ambiguous, the system combines internal and external knowledge to improve robustness, after which the generator receives the query together with the context selected in the foregoing steps. This approach consists of two variations, specifically CRAG and Self-CRAG, with Self-CRAG incorporated the idea of Self-RAG [32]. Mallen et al. assessed the effectiveness of this approach by using major benchmark datasets including PopQA [33] for short form production, Biographical [34] for long form generation, PubHealth [35] for true-false questions, and Arc-Challenge [36] for multiple choice. The study reported accuracy scores for CRAG and Self-CRAG using LLAMA2-fh-7b on PopQA, Biography, PubHealth, and Arc-Challenge datasets (see Table 1). The best performance was achieved by Self-CRAG with accuracy of 69.1 on Biography dataset. The disadvantages of CRAG are dependency on retrieval evaluators and potential bias in web searches.

The method named LeanContext was proposed by Arefeen et al. [37]. LeanContext methodology seeks to minimize the size of a context while maintaining accuracy by utilizing the original context. Experimental LeanContext studies were performed on multiple datasets including the Arxiv dataset [38], BBC dataset [39] and NarativeQA [40]. The LeanContext framework utilizes the ROUGE method [41] to evaluate its performance. The highest ROUGE score for the Arxiv dataset is 0.2305, 0.5017 for the BBCNews, and 0.5244 for the NarrativeQA dataset. The effectiveness of LeanContext, however, relies heavily on the proper training of the reinforcement learning agent, which can be time consuming.

The Rewrite-Retrieve-Read paradigm was introduced by Ma et al. [17]. Rewrite first converts the original query into one or more specific search queries. These queries are then passed to the Retrieve phase, where they are sent to the document index to fetch relevant articles that supply factual context for generation. Finally, in the Read phase, the LLM receives the relevant documents and the search query and

generates an answer to the user's original query. Training the rewriter happens in two stages. First step is Warm-up: asks a large model such as GPT-4 to rewrite training questions into sharper search queries. They keep only the question–query pairs that lead to correct answers and use these ''pseudo-labels'' to fine-tune a smaller rewriter model (T5-large, 770 million parameters) so it can copy the same query style. Second step is Reinforcement learning: Once the rewriter is reasonably skilled, it starts writing its own queries. Each query goes to the retriever; the retrieved documents are passed to a reader LLM, which produces an answer. The system gives a reward based on answer accuracy and adds a KL-divergence penalty if the rewriter's style drifts too far from the original model. Using Proximal Policy Optimization (PPO), the rewriter's parameters are updated so it more often creates queries that steer the reader toward the correct answer. Multiple datasets, including HotpotQA [42], AmbigNQ [43], PopQA [33], MMNU [44], were used to evaluate the suggested approach. The highest Exact Match score of 45.72 was obtained for PopQA dataset. An inherent limitation of the rewrite-retrieve-read approach is the issue of balancing generalization and specialization.

The approach developed by Shao et al. [27] was called ITER-RETGEN. It involved iterating over retrieval-augmented generation and generation-augmented retrieval. The outputs of retrieval-augmented generations were employed to provide useful context for generation-augmented retrieval. This approach offered the benefits of operational efficiency and enhanced performance.

Sun et al. [45], proposed a method known as the Recite-Read method. Recitation is carried out by giving the LLM examples of query–support pairs inside the prompt. From these samples, the model learns in context to recall the relevant paragraph when it sees a new question, so it can work without a search engine. Yet a single fact may appear in several corpora. To avoid depending on one paragraph only, the method generates several recitations through sampling and forms a draft answer for each. The system then chooses the final answer by majority vote. This strategy lowers the chance of error caused by faulty recitation. The process is known as Recite-Read. Multiple benchmarks, including Natural Question [46], Trivia Question [47] and HotpotQA [42], are employed to evaluate this approach in comparison to direct questions. Updates are expensive, which is the primary disadvantage of the Recite-Read method.

Deng et al. [48], proposed a method called Credibility-Aware Attention Modification (CrAM). CrAM is a method that makes RAG systems aware of source credibility. It first identifies the attention heads in the LLM that most strongly influence the model when it accidentally relies on low-credibility documents to generate incorrect answers, and then rescales those heads by multiplying each token's attention weight by the credibility score of its source document—trusted sources keep their full weight, while doubtful ones receive less. This adjustment steers the model toward reliable information and away from possible misinformation, improving answer accuracy without the need to retrain the entire LLM. This approach was evaluated using Trivia Question [47]. Effectiveness of the CrAM approach lies in its user-friendly plug-and-play idea, which enhances its usability. However, the limitation of CrAM is the granularity of the credibility score.

Wang et al. [21] introduced a technique named Relevance-Aware Retrieval-Augmented (REAR) with the goal of developing a system that possesses a high level of self-awareness on the applicability of resources in RAG systems. REAR first operates by concatenating the user query with each document and feeding the pair into a lightweight LLM, which computes a relevance score indicating how closely the document matches the query. This score is converted into a guidance vector that is injected into the prompt: when the score is high, the corresponding document is forwarded to the generator; when it is low, the LLM relies on its internal knowledge to produce the answer. For noise robustness, the model is trained jointly on a binary match/no-match signal and a ranking objective that orders documents by relevance, and it is further exposed to noisy documents so that it can still answer correctly even under perturbations. Evaluation of the REAR approach was conducted on many datasets including Natural Questions [46], TriviaQA [47], WebQuestion [49], dan SquaD [50]. While REAR offers benefits in terms of effective training methods and robustness to external retrieval of different retrieval capacities, the drawback is the computational complexity and lack of granularity in attention.

Glass et al. [51] proposed a new approach called Re2G (Retrieve, Rerank, Generation). It works in three steps. First, a dual-retrieval stage uses Dense Passage Retrieval (DPR) and BM25 to gather 100 candidate documents. Next, a BERT cross-encoder reranks these candidates by scoring how well each document matches the query and keeps the top-k pieces of evidence. Finally, the chosen passages are passed to a BART-large generator, which produces the answer. The authors tested Re2G on several datasets, including T-REx, Natural Questions, FEVER, and Wizard of Wikipedia. The method improves the accuracy of document selection through the reranker and remains modular, but the extra reranking step also adds latency.

Methods such as Dartboard [25] and REAR [21] achieved strong performance, however introduced high computational overhead, limiting their practicality in real-world applications. Similarly, methods like CRAG [26] and ITER-RETGEN [27] depend heavily on retrieval accuracy, which can reduce their effectiveness in handling diverse queries.

III. CONTEXT AWARE RETRIEVAL AUGMENTED GENERATION

This section is divided into two subsections. The first subsection explains RAG, focusing on its mechanism and working principles. The second subsection introduces CA-RAG, which bypasses the retrieval process before

TABLE 1. Summary of previous research addressing challenges in Retrieval-Augmented Generation (RAG) and related tasks.

Paper	Datasets	Metrics	Proposed Method	Result
[25]	Retrieval-Augmented Generation Benchmark (RGB)	Exact String Match (ESM), Normalized Discounted Cumulative Gain(NDCG)	Dartboard	Metric: RGB ESM: 85.6 NDCG: 0.975
[26]	PopQA, Biography, PubHealth, Arc-Challenge	Accuracy	CRAG, Self-CRAG	Metric: Accuracy Method: CRAG / Self-CRAG PopQA: 39.8 / 49.0 Bio: 47.7 / 69.1 Pub: 9.1 / 0.6 Arc: 25.8 / 27.9
[37]	Arxiv, BBCNews, NarrativeQA	ROUGE score	LeanContext	Metric: ROUGE Arxiv: 0.23 BBC: 0.50 Narrative: 0.52
[17]	PopQA, AmbigNQ, HotpotQA, MMNU	Exact Match (EM), F1	Rewrite-Retrieve-Read	Metric: EM, F1 HotpotQA: 34.38, 45.97 AmbigNQ: 47.8, 60.71 PopQA: 45.72, 49.51 MMNU: 43.2, –
[27]	HotpotQA (HP) 2WikiMultiHopQA (MH) MuSiQue (M) Bamboogle (B) Feverous (F) StrategyQA (S)	Exact Match (EM), F1, Acc+	ITER-RETGEN	Metric: EM, F1, Acc+ HP: 45.1, 60.4, 72.9 MH: 33.5, 47.4, 58.4 M: 26.1, 42.0, 41.0 B: 40, 50.7, 60.8 F: –, –, 70.5 S: –, –, 74.1
[45]	Natural Question, Trivia, HotpotQA	Exact Match (EM), F1	Recite Read	Metric: EM, F1 HotpotQA: 37.11, 48.37 Trivia: 83.5, 88.03 NQ: 35.84, 49.12
[48]	TriviaQA	Exact Match (EM), F1	Credibility-aware Attention Modification (CrAM)	Metric: EM, F1 Trivia: 36.9, 48.45
[21]	Natural Questions, TriviaQA, WebQuestions, SQuAD	JAcc, Hit, Exact Match (EM), F1	REAR	Metric: JAcc, Hit, EM, F1 NQ: 74.85, 68.25, 53.32, 61.55 Trivia: 83.28, 77.34, 70.46, 78.07 WebQ: 74.12, 57.93, 42.24, 52.54 SQuAD: 65.26, 60.83, 41.88, 53.47
[51]	TREx, Natural Question, Trivia QA, FEVER Wizard of Wikipedia	Accuracy, F1	Re2G	Metric: Acc, F1 TRex: 86.6, 89.2 NQ: 46.7, 62.4 Trivia: 74.1, 80.86 Fever: 91.05, - Wiki: -, 19.35

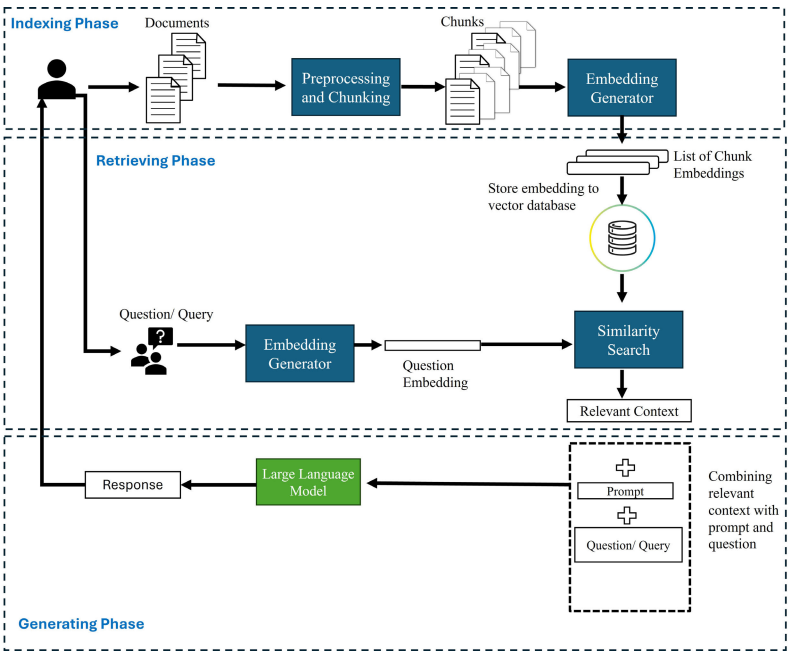

FIGURE 1. Traditional retrieval augmented generation workflow.

generation and emphasizes a post-processing approach. This method is designed to address the limitations of conventional RAG, such as computational complexity and reliance on the retrieval process.

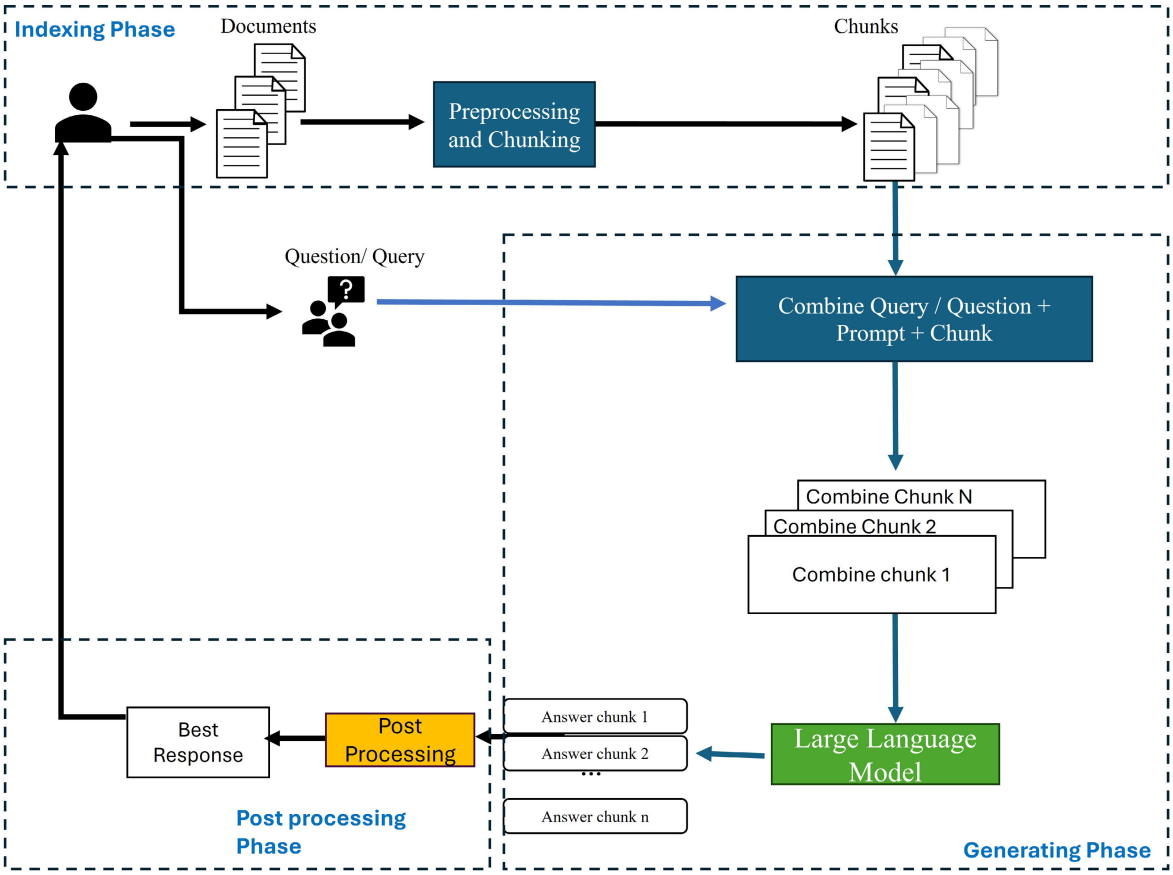

FIGURE 2. Context aware retrieval augmented generation.

A. RETRIEVAL AUGMENTED GENERATION

RAG works in three main stages: indexing, retrieving, and generating. Figure 1 represent the RAG workflow.

In the indexing stage, documents are split into smaller chunks, which are transformed into vector representations using an embedding function. In this study, we used Sentence-BERT [52], a transformer-based model that is designed to produce sentence-level embeddings with high semantic accuracy. These vector representations are saved in a vector database to make retrieval process more efficient in the next steps.

When a user makes a query, the first operation performed by the system is the retrieving stage. In this phase, the question is processed using the same embedding technique as in indexing, converting it into a vector representation. A semantic search using similarity measures (e.g., cosine similarity) is then performed between the vectorized query and the vectors stored in the database to find the most relevant chunks. The similarity between each chunk and the query is calculated, and the results are sorted to select the top-k chunks.

In the generating stage, the retrieved chunks are combined with the user's original question and used as input to a large language model (LLM). The LLM creates a clear and appropriate response, combining the external information from the retrieved documents with its internal knowledge to give accurate and useful answers.

B. CONTEXT AWARE RETRIEVAL AUGMENTED GENERATION

Context Aware Retrieval Augmented Generation (CA-RAG) operates through three main stages: indexing (III-B1), generation (III-B2) and post-processing (III-B3). Fig. 2 provides an overview of the workflow of the proposed CA-RAG system. Unlike traditional RAG, CA-RAG improves the traditional "retrieve and read" approach with adjustments in its stages to better handle context-aware tasks, as better described in the following.

1) INDEXING STAGE

In the indexing stage, documents undergo pre-processing and chunking processes, in a way similar to the one used in the traditional RAG workflow. However, as it is highlighted in [37], traditional chunking often rely on text splitters. While this method can be effective, text splitters sometimes fails to keep the semantic meaning of a document during the chunking process. To solve this problem, a simple semantic chunking method is proposed in this study.

The semantic chunking process, as shown in Algorithm 1, starts by converting the text (T) into a list of sentences (S).

Algorithm 1 Semantic Chunking

```
Input: Text T, Threshold θ
Output: List of Chunks C
S ← Split_Sentences(T) {Split text into sentences}
C ← [] {Initialize list of clusters}
CurrentCluster ← [] {Initialize current cluster}
E_prev ← NULL {Initialize previous embedding}
for each sentence s ∈ S do
    E ← Embed(s) {Convert sentence into embedding}
    E_normalized ← E/||E|| {Normalize embedding}
    if E_prev ≠ NULL then
        Similarity ← E_normalized · E_prior {Compute cosine similarity}
        if Similarity < θ then
            C ← C ∪ {CurrentCluster} {Store current cluster}
            CurrentCluster ← [s] {Start new cluster}
        else
            CurrentCluster ← CurrentCluster ∪ {s} {Add to current cluster}
        end if
    else
        CurrentCluster ← CurrentCluster ∪ {s} {Add first sentence to cluster}
    end if
    E_prev ← E_normalized {Update previous embedding}
end for
C ← C ∪ {CurrentCluster} {Store final cluster}
return C =0
```

Each sentence is then transformed into an embedding (E) using the spaCy library [53]. The embeddings are normalized by dividing each E by its Euclidean norm ($||E||$), which can be seen in the following equation:

$$E_{\text{normalized}} = \frac{E}{||E||} \tag{1}$$

The similarity between the current normalized embedding ($E_{\text{normalized}}$) and the previous one (E_{prior}) is calculated to find their similarity:

$$\text{Similarity} = E_{\text{normalized}} \cdot E_{\text{prior}} \tag{2}$$

Here, $E_{\text{normalized}} \cdot E_{\text{prior}}$ shows the dot product of the two normalized vectors. If the similarity value is below a predefined threshold (θ), the sentence is stored in a new cluster (C). Otherwise, it is added to the current cluster. This iterative process continues until all sentences are processed, creating semantically coherent chunks.

The generated chunks become the input for the next stages. Unlike traditional RAG workflows (that use algorithms such as sparse or dense retrievers in the retrieving stage), the proposed method skips the retrieval step and goes directly to the generation stage. By skipping retrievers, the approach reduces complexity of this phase, while maintaining better contextual relevance.

2) GENERATION STAGE

In the generation stage, each context chunk (c) is combined with a prompt (p) and the question (q), producing n chunk-based answers. These answers are generated using Zephyr [54], which was chosen for its ability to generate context-aware responses efficiently. The generated chunk-based answers are then passed into the post-processing stage.

3) POST-PROCESSING STAGE

In the post-processing stage, two different techniques are used. Both techniques use a similarity function (either cosine similarity or dot product) to evaluate how relevant each answer is with respect to the context. The first technique (named as Technique 1 in the following) identifies the best answer, a^*, based on the similarity between the generated answer and the question (i.e., by taking the highest similarity score):

$$a^* = \arg\max_{a_i \in A} \left(\text{sim}(a_i, Q)\right) \tag{3}$$

where a^* is the best answer, $A = \{a_1, a_2, \ldots, a_n\}$ is the set of given answer, Q is the question and $\text{sim}(x, y)$ denotes a similarity function instantiated as either cosine similarity or dot product.

The second technique (named as Technique 2 in the following) removes answers which are not relevant: if the similarity between a generated answer and its context is below a threshold (θ), the system either marks the answer as "not relevant" and remove it completely. In this way, the set of relevant answers is obtained:

$$A_{\text{filtered}} = \{a_i \in A \mid \text{sim}(a_i, C) \geq \theta\} \tag{4}$$

$$\forall a_i \in A, \text{ if } \text{sim}(a_i, C) < \theta, \ a_i \notin A_{\text{filtered}} \tag{5}$$

Subsequently, the generated answer with the highest similarity score is chosen as the best answer, a^*:

$$A_{\text{sorted}} = \text{Sort}(A_{\text{filtered}}, \text{sim}(a_i, C), \text{descending}) \tag{6}$$

$$a^* = \arg\max_{a_i \in A_{\text{filtered}}} \left(\text{sim}(a_i, C)\right) \tag{7}$$

where $A = \{a_1, a_2, \ldots, a_n\}$ is the set of all generated answers, C is the context, θ is the similarity threshold, and A_{filtered} is the set of filtered relevant answers. Both these post-processing techniques have been considered for evaluation (as part of the CA-RAG setup described in point 2) of Section IV-D). However, only Technique 2 (i.e., CA-RAG with similarity calculated between generated answer and context/chunk) has been considered in the presentation of the experimental results. The rationale behind the adoption of the only Technique 2 relies on the fact that, since the LLM generates each candidate answer conditioned on both the question and the context chunk, we may assume that the user intent is already included and taken into account in the LLM answer generation. In addition, Technique 2 acts as a filter for hallucinations, thus managing also out of context answers, as a conservative abstention

which is useful in specialized domains where the accuracy and completeness of the answers are of pivotal importance. Actually, we finally observed that Technique 2 consistently yielded better performance than Technique 1.

IV. EXPERIMENTS AND RESULT

In this study, we conducted experiments to evaluate the performance of the proposed CA-RAG method in two different scenarios. The first scenario used the TriviaQA [47], Natural Question [46], AmbigQA [43], and Stanford Question Answering Dataset (SQuAD) [50] benchmark dataset, while the second scenario involved a custom dataset with questions divided into general, specific, and out-of-context categories. We made a comparative evaluation among our proposed approach and multiple RAG configurations using different retrieval setups: RAG with MiniLM as embedding model, RAG with Contriever as embedding model, and additional two configurations applying a Reranker to both of the two aforementioned embedding setups.

This section is organized into five subsections to give a clearer explanation of the process and findings.

(A) Evaluation Methods provides a description of the adopted evaluation metrics.
(B) Dataset Preparation explains the source datasets and the pre-processing steps that led to the creation of the dataset that has been used in the presented experiments.
(C) Traditional RAG Setup describes the experimental setup used for RAG, including the maximum chunk size, the length of generative tokens, and the pretrained model used.
(D) CA-RAG Setup describes the experiment settings used in the creation of CA-RAG, both from the post processing performed on both scenarios.
(E) Experimental Result presents the outcomes in tables and figures, making it easier to understand and also interpreting the results and how they relate to the research objectives.

A. EVALUATION METHODS

In this study, different evaluation methods have been used for the two experimental scenarios, in order to assess the performance of the proposed approach.

For Scenario 1, we use BERTScore [55] as the main metric. BERTScore evaluates the similarity between the generated answer and the ground-truth answer by comparing their contextual embeddings. It calculates Precision, Recall, and F1-score using a pre-trained language model as its backbone, ensuring a semantic understanding of the text instead of just matching exact tokens.

In Scenario 2, the evaluation focuses on the system's ability to handle specific, general, and out-of-context questions under more realistic conditions, that is including real documents, such as scientific articles, in order to assess the capability of the systems of handling much longer contextual information, where only a part of it may be relevant to specific questions. Unlike Scenario 1, we used a manual evaluation method in this case.

In Scenario 2, the evaluation was carried out at the chunk level and the question level. This means that human evaluators assessed whether the system generated a correct or incorrect answer based on the result of the generated answer obtained from the LLMs model, according to several conditions. As a result, for each topic with n questions and m retrieved chunks, the evaluation required assessing $n \times m$ question-chunk pairs. Each pair was evaluated individually, making the assessment more fine-grained and sensitive to context.

The manual evaluation for Scenario 2 followed these criteria: a generated answer which is found to be correct for specific or general questions has been classified as a True Positive (TP); when the system provides an incorrect answer for specific and general questions, this is classified as a False Positive (FP); the case in which the system correctly responds with "no context provided" for out-of-context questions is classified as a True Negative (TN); finally, if the system provides an answer different from "no context provided" for out-of-context questions, this is classified as a False Negative (FN).

The choice of producing "no context provided" as an answer to out-of-context questions has been made in order to handle those cases where the LLMs generate answers based on their internal knowledge, which may lead to hallucinations, and this frequently occurs in domain-specific scenarios, where general knowledge provided by the LLM may be inadequate or even inaccurate. Therefore, we intentionally adopted the conservative strategy of returning "no context provided" when the retrieved context does not contain relevant or sufficient evidence to answer the query. TP, FP, TN and FN metrics were finally used to compute overall Accuracy, Precision, Recall, and F1-score. To ensure reliability and consistency, the manual evaluation has been conducted by 1–3 domain experts, providing a thorough and expert-level assessment of the system's performance.

B. DATASET PREPARATION

In this study, we used two types of datasets for the two evaluation scenarios. In the first scenario, we used several benchmark datasets adopted for the assessment of many RAG solutions, which are **TriviaQA** [47], **SQUAD** [50], **Natural Question** [46] and **AmbigQA** [43]. TriviaQA includes Question Answer pairs with passages taken from web sources or Wikipedia. SQuAD is a dataset built through crowdsourcing. The dataset was generated from Wikipedia articles. The Natural Question dataset gathers real questions typed into Google Search and matches them with the resulting Wikipedia pages. AmbigQA is a dataset created to assess ambiguity in the open question domain. The experiments have been conducted using subsets of the above-mentioned four established QA benchmark datasets, for a total benchmark dataset size of about 10 thousand question/answer pairs.

The selection of these four datasets is based on specific considerations regarding the characteristics and challenges inherent in the Question Answering task. The SQuAD dataset has been chosen because it focuses on explicit answers within short contexts, where the answer can be found as a text span from the provided paragraph. The TriviaQA dataset serves a similar function to SQuAD, being explicit in answer retrieval; however, it differs in that it requires cross-sentence reasoning to obtain the answer, thus involving a larger context. The Natural Questions dataset has been selected because it demands deep comprehension and advanced reasoning skills, extending beyond mere span matching as seen in SQuAD or cross-sentence matching in TriviaQA. The final dataset, AmbigQA, concentrates on semantically ambiguous questions that may have multiple valid answers.

In the second scenario, we used a custom dataset consisting of several scientific documents [56], [57], [58], [59], [60]. The evaluated papers have been published in several reputable sources for peer-reviewed publications such as IEEE, Springer, and Elsevier and can be accessed through [56], [57], [58], [59], [60]. The Scenario 2 dataset is divided into five main topics, each representing a specific document as follows:

- *Bike sharing predictions* [56],
- *Landslide prediction* [60],
- *Reputation assessment* [59],
- *Scenario editor* [57], and
- *Thermal camera for people counting* [58].

The rationale behind the design of this custom dataset for Scenario 2 aims to reflect many real-world scenarios and use cases, where the document collections are of small or moderate size, covering domain specific knowledge in which answer accuracy and completeness are paramount. For each topic, a set of questions has been defined, as explained in the following: the set is composed of 3 questions which are specific for the particular topic. In addition, there are 3 general questions (i.e., related to all the five selected topics) and 3 not relevant questions (out-of-context questions). This approach has been followed in order to test the model's performance in handling requests that are not related to the context. Each question, both general and specific, was developed based on several considerations. For the specific questions, an extractive approach was used, where each question refers to a particular section of the text and the answer can be found directly within the passage without requiring additional reasoning. In contrast, the general questions follow an abstractive approach, where the questions are formulated more broadly, and answers may either be directly stated in the article or require a higher level of inference. As for the out-of-context questions, they were derived from everyday general knowledge and trivia, for example: "what is the color of a panda?".

In the pre-processing stage, several steps are performed to improve the quality of the used dataset. The first step is removing escape sequences, where special characters like `\n` or `\\n` are replaced with spaces. This ensures the text readability is not disturbed by unnecessary symbols. Next, extra spaces between words are removed. Backslashes (`\`) and slashes (`/`) are also removed. Repeated characters, such as punctuation marks like "!!!" or "...", are replaced with a single instance.

In addition, non-breaking spaces like `\xa0` are replaced with regular spaces, which helps clean invisible special characters. Square brackets, such as `[` and `]`. Hyphenated words split with spaces are corrected and joined back to form a single word. Lastly, escape characters are cleaned again if required, ensuring the text is free of all formatting issues.

C. RAG SETUP

This research applies several important rules to ensure the efficiency and relevance of the process. First, in the *chunking* process, we limit the maximum number of tokens to 512 tokens. Second, for the answers generated during the generation stage, we restrict the length to **100 tokens**. For the embedding process, we utilize two different models: S-BERT [52] with the *all-miniLM-L6-V2* model and *Contriever-Msmarco* [61]. Similarly, for generating the answers, we use *Zephyr-7b*.

D. CA-RAG SETUP

As earler mentioned, two experimental scenarios has been designed in this study, in order to evaluate different aspects of the proposed approach. Scenario 1 focus on testing the system using the benchmark dataset TriviaQA [47], **SQUAD** [50], **Natural Question** [46] and **AmbigQA** [43]. This scenario aims at evaluating multiple setups and identify the most effective scheme to improve the quality of generated answers.

The CA-RAG configurations used in the experiments are the following:

1) List Context-Aware RAG (List CA-RAG), which averages the F1, Precision, and Recall scores across all evaluated chunks
2) List CA-RAG applying a post-processing method using similarity methods (either cosine similarity or dot product), which has been implemented following two main steps: first, similarity is calculated between the generated answer and the related context; then, for each question, the answer with the highest similarity score is selected. For this setup, we have used two different embedding models, i.e.: MiniLM and Contriever, in order to evaluate wether our approach is robust or sensitive to different embedding models.

In Scenario 2 we applied the post-processing method to evaluate the robustness of the system in handling also out-of-context questions with a custom, domain specific dataset. If the similarity value was lower than a set threshold, then the generated answer was marked as "no context provided". In this scenario, the generated answers were also evaluated manually by 1–3 domain experts to assess their accuracy and

relevance. Subsequently, the same evaluation was conducted as previously described.

E. EXPERIMENTAL CONFIGURATION

In this section, we describe all the different configurations used in this study. The configurations include the generator model, context length, framework, hardware, chunking method, chunk length, output length, benchmark dataset, preprocessing steps, comparison methods, and post-processing (specifically for the proposed method). Zephyr-7B-Beta, with a context length of 4096 tokens, has been used for text generation. The experiments were conducted using several libraries such as Hugging Face and PyTorch, and were supported by two NVIDIA RTX 4090 GPUs (each with 24GB of memory).

Document processing was carried out using two chunking methods: semantic chunking, as previously discussed, and fixed-size chunking. The maximum chunk length is set to 512 tokens, and the generation output from the LLM model is limited to 512 new tokens. On semantic chunking, we used $\theta = 0.3$. We selected this threshold for similarity methods because, empirically, sentence embedding models show that paraphrase pairs usually yield high similarity values ($\approx 0.7-0.9$), while unrelated pairs tend to approach 0. Therefore, by setting θ near this lower range, the segmentation allows for greater diversity between chunks and prevents sentences with only weak semantic connections from being forced into the same cluster. The benchmark datasets used for experiments in Scenario 1 include TriviaQA, SQuAD, NaturalQA, and AmbigQA. Before being used, the data went through several preprocessing steps, such as cleaning escape characters, removing special characters, and normalizing spacing and punctuation. In addition, a post-processing step was applied specifically to the CA-RAG variant, using similarity thresholding with a threshold value of $\tau = 0.6$. The choice of 0.6 is based on empirical experiments, which showed that this value produced the best results. The table 2 summarize the experimental configuration.

TABLE 2. Experimental configuration summary.

Component	Value / Description
Generator Model	Zephyr-7B-Beta (HuggingFace)
Context Length	4096 tokens (default)
Framework	Hugging Face Transformers, PyTorch
Hardware	2 × NVIDIA RTX 4090 (24GB)
Chunk Method	Semantic (embedding-based) and Fixed
Chunk Length (Semantic)	`max_chunk_length = 512`
Chunk Length (Fixed)	`chunk_length = 512`
Output Length	`max_new_tokens = 512`
Benchmark Dataset	TriviaQA, AmbigQA, SQuADv2, NaturalQA
Preprocessing	Cleaning escape sequences, removing special characters, normalizing spacing & punctuation
Comparison Method	RAG - MiniLM embedding, RAG - Contriever embedding, RAG - MiniLM embedding and Rerank, RAG - Contriever embedding and Rerank
Post-processing (CA-RAG)	similarity thresholding with $\tau = 0.6$ for context answer alignment

F. EXPERIMENTAL RESULTS

This section presents the results of the experiments conducted to evaluate the performance of the proposed CA-RAG approach compare to Traditional RAG. The experiments have been designed to assess different post-processing methods in Scenario 1, and to evaluate the system's robustness using a custom dataset in Scenario 2. The performance metrics used includes Precision, Recall, F1-score and Accuracy.

In this study, we compare our proposed method against RAG with MiniLM as embedding model [16], RAG with Contriever as embedding model [61], and further two configurations adding a Reranker [62], [63] to both of the two aforementioned embedding setups. We also experimented two different chunking strategies: fixed-size chunking and semantic chunking. Additionally, for Scenario 2, we compared also our method with a long-context QA system using Llama 3.3-70B (with 128k tokens of context length) for text generation. This has been done to investigate and compare the capabilities in QA tasks of larger LLMs that can process full context in a single pass, without the need to implement the chunking phase.

Table 3 presents the results obtained in Scenario 1, where we evaluated our models on several widely used benchmark datasets. Two chunking strategies were examined: semantic chunking and fixed-size chunking. In general, the proposed method excels on datasets with longer contexts. On TriviaQA, for example, it achieves the best overall performance. Actually, our CA-RAG setup with miniLM embeddings and post-processing strategy using dot product yields an F1 score of 0.8629 (with semantic chunking) and on contriever embedding using dot product achieved 0.9089 (with fixed-size chunking), surpassing all RAG baselines (RAG with MiniLM embeddings, RAG with Contriever embeddings, RAG with MiniLM embeddings and Rerank, and RAG with Contriever embeddings and Rerank).

For the more ambiguous AmbigQA benchmark, our approach remains competitive in terms of recall and F1 when fixed-size chunks are used; however, in the semantic setting the top scores are obtained by RAG models equipped with Contriever embeddings (with and without Reranker). Finally, on shorter-context datasets such as NaturalQA and SQuAD, our model is comparatively less effective, whereas the RAG-Rerank baseline achieves the strongest results under both chunking schemes.

In Scenario 1 we saw a clear split: RAG variants performed best on short-context datasets (e.g.,NaturalQA,SQuAD), while CA-RAG led on the longer-context TriviaQA benchmark. To confirm that CA-RAG's advantage truly scales to long documents, Scenario 2 evaluates scientific papers that contain much larger passages of specialised text and includes the long-context model Llama-3 as an additional baseline.

The results in Table 4 indicate that CA-RAG achieves the highest F1 scores under both semantic and fixed-size

TABLE 3. Results of experiments in scenario 1 for the following benchmark datasets: TriviaQA, NaturalQA, SQuAD and AmbigQA (total benchmark dataset size of about 10 thousand question/answer pairs). Our proposed method, CA-RAG is shown in bold in the table.

Chunking Method	Method	Embedding Model	Trivia QA			Natural QA			SQuAD			Ambig QA		
			Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1	Prec.	Rec.	F1
Semantic	RAG	miniLM	0.8536	0.8158	0.8334	0.8737	0.8448	0.8582	0.8852	0.8465	0.8649	0.8311	0.8069	0.8179
	RAG	Contriever-Msmarco	0.8511	0.8338	0.8413	0.8909	0.8970	0.8930	0.8964	0.9130	0.9039	0.8413	0.8453	0.8420
	RAG-Rerank	miniLM	0.8545	0.8382	0.8452	0.8914	0.8974	0.8934	0.8972	0.9141	0.9049	0.8399	0.8464	0.8419
	RAG-Rerank	Contriever-Msmarco	0.8545	0.8383	0.8453	0.8914	0.8974	0.8934	0.8971	0.9140	0.9048	0.8400	0.8465	0.8420
	CA-RAG without post processing	miniLM	0.8443	0.8185	0.8303	0.8127	0.8037	0.8076	0.8169	0.8179	0.8170	0.8023	0.7911	0.7960
	CA-RAG with post processing Cosine Similarity	miniLM	0.8547	0.8235	0.8380	0.8827	0.8561	0.8684	0.8958	0.8578	0.8759	0.8342	0.8129	0.8226
	CA-RAG with post processing Cosine Similarity	Contriever-Msmarco	0.8449	0.8829	0.8628	0.8571	0.8840	0.8695	0.8580	0.8969	0.8764	0.8141	0.8364	0.8242
	CA-RAG with post processing Dot product	miniLM	0.8454	0.8826	0.8629	0.8571	0.8840	0.8695	0.8580	0.8969	0.8764	0.8141	0.8364	0.8243
	CA-RAG with post processing Dot product	Contriever-Msmarco	0.8583	0.8693	0.8628	0.8567	0.8566	0.8558	0.8599	0.8717	0.8651	0.8280	0.8359	0.8309
Fixed-Size	RAG	miniLM	0.8404	0.8206	0.8304	0.8822	0.8848	0.8835	0.8792	0.8893	0.8842	0.8417	0.8356	0.8386
	RAG	Contriever-Msmarco	0.8404	0.8206	0.8304	0.8822	0.8848	0.8835	0.8792	0.8893	0.8842	0.8417	0.8356	0.8386
	RAG-Rerank	miniLM	0.8594	0.8372	0.8471	0.9071	0.9227	0.9141	0.8851	0.9009	0.8929	0.8553	0.8538	0.8533
	RAG-Rerank	Contriever-Msmarco	0.8594	0.8371	0.8470	0.9071	0.9227	0.9141	0.8972	0.9009	0.8981	0.8553	0.8538	0.8533
	CA-RAG without post processing	miniLM	0.8594	0.8738	0.8654	0.8721	0.8847	0.8774	0.8698	0.8957	0.8819	0.8229	0.8415	0.8312
	CA-RAG with post processing Cosine Similarity	miniLM	0.8951	0.9084	0.9007	0.8842	0.8986	0.8904	0.8817	0.9107	0.8953	0.8508	0.8723	0.8605
	CA-RAG with post processing Cosine Similarity	Contriever-Msmarco	0.8950	0.9087	0.9008	0.8841	0.8986	0.8903	0.8815	0.9108	0.8952	0.8496	0.8721	0.8597
	CA-RAG with post processing Dot product	miniLM	0.8870	0.8943	0.8896	0.8506	0.8465	0.8477	0.8517	0.8599	0.8550	0.8473	0.8592	0.8522
	CA-RAG with post processing Dot product	Contriever-Msmarco	0.9075	0.9120	0.9089	0.8700	0.8726	0.8704	0.8756	0.8891	0.8816	0.8609	0.8714	0.8652

chunking, outperforming every other RAG configuration as well as the Llama-3 baseline used for comparison with long-context LLMs. This advantage stems from CA-RAG's built-in post-processing filter, which discards irrelevant candidate answers before selecting the correct ones. Because we report metrics at the chunk level, we can directly observe how effectively CA-RAG identifies pertinent segments and produces accurate responses. Crucially, it attains this accuracy while processing a much shorter context window than Llama-3, demonstrating that careful chunk-answer selection can match and even surpass the performance of long-context models.

As additional context, Table 4 shows that, in both traditional RAG variants with MiniLM and Contriever embeddings, semantic chunking works better than fixed-length chunking; this may be due to the fact that it chooses passages that closely match to previous sentences to form a chunk, leading to more similar topics in chunk, while fixed-size chunks can mix several different topics in one chunk, as well as splitting textual contents that may be contextually related in separated chunks, thus making the retrieved information potentially less relevant and lowering overall performance. When a reranking step is added to traditional RAG configurations, the ones using fixed-size chunks suddenly perform much better: the reranker can spot and push the query-relevant sentences back to the top even if they sit inside cluttered chunks, eliminating the fixed method's weakness. Semantic chunks, however, gain only a small boost because their passages were already tightly aligned from the start.

In real-world case studies, the documents used often differ from those in benchmark datasets. Real-world documents typically contain more complex and extensive contexts, such as in technical manuals, complete books, or scientific articles. Under these conditions, the RAG system frequently

TABLE 4. Results of experiments in scenario 2: Performance of various RAG-based models and long context QA (using Llama3.3-70B) evaluated on a custom dataset composed of scientific documents. Our proposed method, CA-RAG is shown in bold in the table.

Method	Embedding Model	Semantic Chunking				Fixed-Size Chunking			
		Acc.	Prec.	Rec.	F1	Acc.	Prec.	Rec.	F1
RAG	miniLM	0.4900	0.5500	0.5900	0.5700	0.4286	0.1481	0.8000	0.2500
RAG	Contriever-Msmarco	0.4762	0.4615	0.6	0.5218	0.4286	0.1481	0.8000	0.2500
RAG-Rerank	miniLM	0.6191	0.6667	0.72	0.6923	0.7619	0.6923	0.900	0.7826
RAG-Rerank	Contriever-Msmarco	0.5476	0.5926	0.6667	0.6275	0.7619	0.6923	0.900	0.7826
CA-RAG without post processing	miniLM	0.4600	0.3400	0.6900	0.4500	0.6092	0.4435	0.9373	0.6021
CA-RAG with post processing (Cosine Similarity)	miniLM	0.9200	0.8700	1.0000	0.9300	0.8539	0.7821	0.9985	0.8771
CA-RAG with post processing (Cosine Similarity)	Contriever-Msmarco	0.9144	0.8935	0.9934	0.9408	0.9297	0.8871	1.0000	0.9400
CA-RAG with post processing (Dot product)	miniLM	0.9704	0.9557	1.0000	0.9773	0.7126	0.5925	0.9617	0.7732
CA-RAG with post processing (Dot product)	Contriever-Msmarco	0.9832	0.9748	1.0000	0.9872	0.7782	0.6827	0.9780	0.8041
Long Context QA with Llama3.3	**All Context (no chunking)**								
	Acc.		Prec.		Rec.		F1		
	0.7619		0.6667		0.9474		0.7824		

produces hallucinated answers. Moreover, RAG models that rely on retrieval may fail to select the most relevant chunk. This issue was observed in Scenario 2, where all the RAG variants — using MiniLM and Contriever as embedding models, as well as with the addition of a reranker — were unable to retrieve the correct chunk, resulting in poor performance. On the other hand, the addition of Rerank occasionally improves the results, as demonstrated in Scenario 1. Nevertheless, in scenario 2, its overall performance remains inferior.

When post-processing is applied to our CA-RAG method, the system evaluates whether the generated answer is related to the given chunk by using dot product or cosine similarity. If the chunk is found to be irrelevant, the system returns a response such as "no context provided". This technique serves as a filter to block out-of-context questions from being answered based purely on the model's memory. By verifying the relevance of the answer to the provided context, the system can detect hallucination answer. This significantly increases the number of true negatives (TN), as the system correctly identifies cases where no valid answer exists in the chunk. At the same time, it reduces false positives (FP), where incorrect answers might previously have been given. As a result, true positives (TP) and false negatives (FN) become more critical, as they reflect the system's ability to correctly answer valid questions. Moreover, as described in Section IV-D, we have used different embedding models in the post-processing phase of the proposed CA-RAG setups, i.e.: MiniLM, and Contriever. Results of this comparative assessment have revealed that Precision, Recall and F1 remained consistent between the two different configurations (as shown in Table 3 and Table 4), therefore proving that our approach is quite robust with respect to the specific choice of the embedding model.

We can observe in Table 4 (related to Scenario 2 experiment results), that CA-RAG with dot-product scoring under fixed-size chunking shows lower precision and F1 with respect to semantic chunking approach (on the same embedding model). This could be due to several aspects: dot product is sensitive to the magnitude of the embedding vectors (unlike cosine similarity), and in our setup we do not normalize embeddings for either the generated answers or the context chunks. In addition, fixed-size chunks can often split relevant information and mix it with not relevant and noisy content. Consistently with these observations, Table 3 (referring to Scenario 1 experiments) shows much less performance variability among our CA-RAG method variants, since the benchmark datasets used (TriviaQA, NaturalQA, SQuAD, AmbigQA) provide usually shorter and more homogenous context, in which the evidence for the answer is typically contained within one or a few contiguous sentences.

To evaluate the effectiveness of our proposed method, we may also qualitatively compare its performance also against several state-of-the-art approaches, as described in the Related Works section. The comparison focused on F1 scores across shared benchmark datasets, namely TriviaQA, NaturalQA, SQuAD, and AmbigQA. Specifically, we referenced CrAM [48], REAR [21], Recite Read [45], and Rewrite-Retrieve-Read [17]. On the TriviaQA dataset, our CA-RAG method with Contriever embedding and fixed-size chunking reports the highest F1 score (0.9007) compared to CrAM (0.4845), REAR (0.7807), Recite Read (0.8803) and Re2G (0.8086). For NaturalQA, our CA-RAG method with MiniLM embedding and fixed-size chunking achieved the best performance in F1 score (0.8904), surpassing REAR (0.6155), Recite Read (0.4912) and Re2G (0.624). A similar trend has been observed on the SQuAD dataset, where

our CA-RAG configuration with MiniLM embedding and fixed-size chunking (0.8953) outperforms REAR (0.5347). Finally, on the AmbigQA dataset, our CA-RAG approach with MiniLM embedding and fixed-size chunking yielded a higher F1 score (0.8605) than the Rewrite-Retrieve-Read method (0.6071), indicating robust performance across diverse QA benchmarks.

TABLE 5. Computational time of the proposed method and traditional RAG (related to Scenario 2).

Chunking Method	Method	Embedding model	Average time per question (s)
Semantic	RAG	miniLM	33.32
	RAG	Contriever-msmarco	33.46
	RAG-Rerank	miniLM	86.36
	RAG-Rerank	Contriever-msmarco	85.81
	CA-RAG	miniLM	262.63
	CA-RAG	Contriever-msmarco	262.65
Fixed-Size	RAG	miniLM	24.02
	RAG	Contriever-msmarco	24.31
	RAG-Rerank	miniLM	240.62
	RAG-Rerank	Contriever-msmarco	233.32
	CA-RAG	miniLM	2289.47
	CA-RAG	Contriever-msmarco	2289.49

G. PROCESSING TIMES ANALYSIS

In this section, a quantitative and comparative evaluation of the computational times for the proposed method is provided, considering Scenario 2 experiments. We selected Scenario 2 because it is more representative of real-world documents. In this context, we provide the average time per question, as shown in Table 5. According to our results, the proposed method records an average running time per question of 262.63s under semantic chunking with the miniLM embedding, and 262.65s with contriever-msmarco. Under fixed-size chunking, the average time per question increases to 2289.47s (miniLM) and 2289.49s (contriever-msmarco), since fixed-size chunking typically yields a larger number of chunks than semantic chunking, and this directly translates into more LLM calls. Compared with the RAG and RAG-Rerank baselines, our approach entails higher computational cost per question. However, our solution mainly addresses use cases and domains with specialized and technical knowledge where answer accuracy, completeness, and faithfulness are often prioritized over minimizing processing times and latency. In such contexts, the additional computational cost can be justified when it reduces hallucinations and improves answers reliability. Moreover, under realistic conditions, a human reader typically needs $\sim$ 70–80s per question to read, locate significant content, and synthesize an answer from a short document (up to 200 words) [64]. Assuming, conservatively, linear scaling with document length, this corresponds to $\sim$ 0.35–0.53s per word; Therefore, since Scenario 2 documents range from a minimum of $\sim$ 6700 to a maximum of $\sim$ 11200 words, the time required to read and provide an answer would be approximately 2345–3920s (i.e., about 39-65 minutes) under the most optimistic rate of 0.35 s/word. This should be considered as a conservative upper bound, which could be appropriate for the first question, while for subsequent questions the reader typically can reuse previously acquired knowledge and only consult specific sections, thus the incremental per-question time could be lower. Our comparison should therefore be interpreted as a worst-case baseline. However, according to this simple model, our system, particularly CA-RAG with semantic chunking (which produced the best overall performances for Scenario 2 with dot-product post-processing), compares favorably to a human baseline for quite large documents, reporting an average of about 262s per question (i.e., about 4 minutes). Therefore, we can conclude that the processing times of our best method remains substantially lower than the time required by a human domain-expert to read specialized documentation and produce similarly accurate and fully grounded answer. Finally, not least, our best method produced significantly higher performances than the best RAG model (RAG with reranking), as highlighted in Table 4 and in Section IV-F, at a slightly higher average computational cost per query (about 262s for CA-RAG with semantic chunking versus 85s for RAG with reranking).

V. CONCLUSION AND FUTURE WORK

This work introduces a novel approach for Retrieval Augmented Generation called Context-Aware Retrieval Augmented Generation (CA-RAG), which adapts the "retrieve and read" framework used by traditional RAG while eliminating vector representations and the top-k process in the retrieve stage. The approach incorporates a similarity evaluator after the generative phase to address hallucination issues and enhance contextual relevance. The choice of either cosine similarity or dot product between the embedding of the generated answer and the context is a reliable proxy to assess chunks relevance. However, this may not always be the case, particularly for queries requiring abstract reasoning or multi-hop answers, as well as when chunks contain both relevant and irrelevant content.

In Scenario 1, we compared our proposed method, CA-RAG, with RAG using MiniLM, RAG using Contriever, and RAG with Rerank applied to both the embedding configurations. We applied both semantic chunking and fixed-size chunking to four well-established benchmark datasets: TriviaQA, NaturalQA, SQuAD, and AmbigQA. We found that different chunking methods led to different results.With semantic chunking, RAG with Rerank achieved better performance than our proposed method. This may be due to a more effective selection of "optimal" chunks, which helps retrieve more relevant answers. In addition, the use of semantic chunking may have played a significant role in helping Rerank perform better in selecting the most suitable chunk for answer generation. However, when using fixed-size chunking, our method outperformed the others. Fixed-size chunking provided better results for our method because it may ensure that each chunk has a consistent length, allowing

the encoder to work more efficiently. In contrast, semantic chunking may produce uneven chunk sizes or split the context in ways that reduce the relevance of retrieved content, which can lower performance on some datasets. On the other hand, Contriever combined with semantic chunking performed better than our method because its retrieval process is more effective at selecting semantically meaningful content to generate answers. In comparison, our List CA-RAG method with either cosine similarity or dot product relies on matching generated responses with context, which may be less precise in semantic alignment.

In scenario 2, the evaluation focused on testing the system's ability to handle specific, general, and outside-the-context questions under more realistic conditions. Unlike Scenario 1, this scenario used a manual evaluation method based on chunk-level assessments and question-level assessment on a custom dataset built with whole scientific papers, thus providing longer and less structured context. For each topic, multiple questions have been paired with retrieved chunks, and human evaluators judged whether the generated answer was correct, incorrect, or appropriately left unanswered when no relevant context was available. The results showed that CA-RAG with post-processing significantly outperformed both RAG and CA-RAG without post-processing, achieving F1-score of 0.9408 on semantic chunking, and 0.94 on fixed chunking. This strong performance is closely related to the method's ability to detect correctly when there is no relevant information in a section, resulting in a high number of True Negatives (TN) and fewer False Positives (FP) and False Negative (FN). In contrast, models without post-processing often produced hallucinated answers, as they lacked a mechanism to filter out irrelevant chunks. These results highlight the importance of post-processing techniques in generative QA, particularly when operating in complex, open-domain environments where questions may not always align with the retrieved content. The experiments also demonstrate that CA-RAG with post-processing outperforms existing methods, making it a reliable solution for Retrieval Augmented Generation tasks. Since our approach provides all the chunks to the LLM without the retrieval stage (as in traditional RAG), it may be most practical for moderated size corpora, in those real-world contexts where answer accuracy and completeness are paramount.

The proposed CA-RAG has shown a higher average per-question time than RAG/RAG-Rerank, with 262.7s under semantic chunking and 2289.5s under fixed-size chunking. The overhead is notably smaller with semantic chunking, which in conclusion can be considered as the most representative approach, since it produced the best overall performances in Scenario 2. Moreover, the processing times of our best CA-RAG configuration remains substantially lower than the time a human expert would require to read specialized documentation and produce similarly accurate answers. Finally, our method produced significantly higher performances in Scenario 2 than the best RAG model (i.e. RAG with reranking), at a slightly higher computational cost per query (about 262s for CA-RAG with semantic chunking, versus 85s for RAG with reranking). For future work, the integration of a chain-of-taught query algorithm, as proposed by [65], will be explored to further enhance the retrieval and generation stages. This could potentially improve precision and reduce hallucinations.

ACKNOWLEDGMENT

The authors acknowledge the support of the PNRR MIUR project CAI4DSA on decisions support systems of FAIR—Future AI Research extended national partnership. A special thanks to the many developers working on the Snap4City (https://www.snap4city.org) and KM4City are open technologies of DISIT Lab.

REFERENCES

[1] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[2] Athropic. (2023). *Introducing Claude*. [Online]. Available: https://www.anthropic.com/news/introducing-claude

[3] A. Grattafiori et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.

[4] K. Singhal et al., "Large language models encode clinical knowledge," *Nature*, vol. 620, no. 7972, pp. 172–180, Aug. 2023. [Online]. Available: https://www.nature.com/articles/s41586-023-06291-2

[5] H.-Y. Zhou, J. N. Acosta, S. Adithan, S. Datta, E. J. Topol, and P. Rajpurkar, "MedVersa: A generalist foundation model for medical image interpretation," 2024, *arXiv:2405.07988*.

[6] S. MacNeil, A. Tran, A. Hellas, J. Kim, S. Sarsa, P. Denny, S. Bernstein, and J. Leinonen, "Experiences from using code explanations generated by large language models in a Web software development E-book," in *Proc. 54th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE)*, vol. 1, Mar. 2023, pp. 931–937. [Online]. Available: https://dl.acm.org/doi/10.1145/3545945.3569785

[7] B. A. Tanay, L. C. Arinze, S. S. Joshi, K. A. Davis, and J. C. Davis, "An exploratory study on upper-level computing students' use of large language models as tools in a semester-long project," in *Proc. ASEE Annu. Conf. Expo.*, Jun. 2024, p. 46557. [Online]. Available: http://peer.asee.org/46557

[8] X. Jia, W. Zhou, X. Sun, and Y. Wu, "EQG-RACE: Examination-type question generation," in *Proc. AAAI Conf. Artif. Intell.*, May 2021, vol. 35, no. 14, pp. 13143–13151. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/17553

[9] B. Zhang, H. Yang, and X. Y. Liu, "Instruct-FinGPT: Financial sentiment analysis by instruction tuning of general-purpose large language models," 2023, *arXiv:2306.12659*.

[10] Y. Yu, H. Li, Z. Chen, Y. Jiang, Y. Li, J. W. Suchow, D. Zhang, and K. Khashanah, "FinMem: A performance-enhanced LLM trading agent with layered memory and character design," in *Proc. AAAI Symp. Ser.*, May 2025, pp. 1–18. [Online]. Available: https://ojs.aaai.org/index.php/AAAI-SS/article/view/31290

[11] Q. Xie, W. Han, X. Zhang, Y. Lai, M. Peng, A. Lopez-Lira, and J. Huang, "PIXIU: A large language model, instruction data and evaluation benchmark for finance," 2023, *arXiv:2306.05443*.

[12] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large language models struggle to learn long-tail knowledge," in *Proc. 40th Int. Conf. Mach. Learn.*, Jul. 2022, pp. 15696–15707.

[13] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.

[14] T. Singh, H. Aditya, V. K. Madisetti, and A. Bahga, "Whispered tuning: Data privacy preservation in fine-tuning LLMs through differential privacy," *J. Softw. Eng. Appl.*, vol. 17, no. 1, pp. 1–22, 2024. [Online]. Available: https://www.scirp.org/journal/paperinformation?paperid=130659

[15] X. Lin, W. Wang, Y. Li, S. Yang, F. Feng, Y. Wei, and T.-S. Chua, "Data-efficient fine-tuning for LLM-based recommendation," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2024, pp. 365–374, doi: 10.1145/3626772.3657807.

[16] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, Dec. 2020, pp. 9459–9474.

[17] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting in retrieval-augmented large language models," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 5303–5315. [Online]. Available: https://aclanthology.org/2023.emnlp-main.322

[18] L. Wang, N. Yang, and F. Wei, "Learning to retrieve in-context examples for large language models," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics*, Mar. 2024, pp. 1752–1767. [Online]. Available: https://aclanthology.org/2024.eacl-long.105/

[19] Y. Wang, P. Li, M. Sun, and Y. Liu, "Self-knowledge guided retrieval augmentation for large language models," in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, Dec. 2023, pp. 10303–10315. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.691/

[20] S. Setty, H. Thakkar, A. Lee, E. Chung, and N. Vidra, "Improving retrieval for RAG based question answering models on financial documents," 2024, *arXiv:2404.07221*.

[21] Y. Wang, R. Ren, J. Li, W. Xin Zhao, J. Liu, and J.-R. Wen, "REAR: A relevance-aware retrieval-augmented framework for open-domain question answering," 2024, *arXiv:2402.17497*.

[22] S. Zeng, J. Zhang, P. He, Y. Liu, Y. Xing, X. Han, J. Ren, C. Yi, S. Wang, D. Yin, and J. Tang, "The good and the bad: Exploring privacy issues in retrieval-augmented generation (RAG)," in *Proc. Findings Assoc. Comput. Linguistics, ACL*, Aug. 2024, pp. 4505–4524. [Online]. Available: https://aclanthology.org/2024.findings-acl.267/

[23] H. Soudani, E. Kanoulas, and F. Hasibi, "Fine tuning vs. retrieval augmented generation for less popular knowledge," in *Proc. Annu. Int. ACM SIGIR Conf. Res. Develop. Inf. Retr. Asia–Pacific Region*, Dec. 2024, pp. 12–22, doi: 10.1145/3673791.3698415.

[24] R. Lakatos, P. Pollner, A. Hajdu, and T. Joó, "Investigating the performance of retrieval-augmented generation and domain-specific fine-tuning for the development of AI-driven knowledge-based systems," *Mach. Learn. Knowl. Extraction*, vol. 7, no. 1, p. 15, Feb. 2025.

[25] M. Pickett, J. Hartman, A. Kumar Bhowmick, R.-u. Alam, and A. Vempaty, "Better RAG using relevant information gain," 2024, *arXiv:2407.12101*.

[26] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," 2024, *arXiv:2401.15884*.

[27] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, 2023, pp. 9248–9274. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.620

[28] F. Y. Loumachi, M. C. Ghanem, and M. A. Ferrag, "Advancing cyber incident timeline analysis through retrieval-augmented generation and large language models," *Computers*, vol. 14, no. 2, p. 67, Feb. 2025.

[29] S. Song, C. Yang, L. Xu, H. Shang, Z. Li, and Y. Chang, "TravelRAG: A tourist attraction retrieval framework based on multi-layer knowledge graph," *ISPRS Int. J. Geo-Information*, vol. 13, no. 11, p. 414, Nov. 2024.

[30] R. Upadhyay and M. Viviani, "Enhancing health information retrieval with RAG by prioritizing topical relevance and factual accuracy," *Discover Comput.*, vol. 28, no. 1, Apr. 2025.

[31] S. Ajay Mukund and K. S. Easwarakumar, "Optimizing legal text summarization through dynamic retrieval-augmented generation and domain-specific adaptation," *Symmetry*, vol. 17, no. 5, p. 633, Apr. 2025.

[32] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," 2023, *arXiv:2310.11511*.

[33] A. Mallen, A. Asai, V. Zhong, R. Das, D. Khashabi, and H. Hajishirzi, "When not to trust language models: Investigating effectiveness of parametric and non-parametric memories," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 9802–9822. [Online]. Available: https://aclanthology.org/2023.acl-long.546

[34] A. Plum, T. Ranasinghe, S. Jones, C. Orasan, and R. Mitkov, "Biographical semi-supervised relation extraction dataset," in *Proc. 45th Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, Jul. 2022, pp. 3121–3130. [Online]. Available: https://dl.acm.org/doi/10.1145/3477495.3531742

[35] N. Kotonya and F. Toni, "Explainable automated fact-checking for public health claims," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 7740–7754. [Online]. Available: https://www.aclweb.org/anthology/2020.emnlp-main.623

[36] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, "Think you have solved question answering? Try ARC, the AI2 reasoning challenge," 2018, *arXiv:1803.05457*.

[37] M. A. Arefeen, B. Debnath, and S. Chakradhar, "LeanContext: Cost-efficient domain-specific question answering using LLMs," *Natural Lang. Process. J.*, vol. 7, Jun. 2024, Art. no. 100065. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S294971912400013X

[38] A. Farhangi, N. Sui, N. Hua, H. Bai, A. Huang, and Z. Guo, "Protoformer: Embedding prototypes for transformers," in *Proc. Adv. Knowledge Discovery Data Mining*. Cham, Switzerland: Springer, 2022, pp. 447–458. [Online]. Available: https://link.springer.com/10.1007/978-3-031-05933-935

[39] P. Singh, P. Kashetty, A. Reddy, G. S. R. Teja, and V. S. Anusha, "Automated news summarization using transformers," in *Proc. Sustain. Advanced Computing*. Cham, Switzerland: Springer, 2024, pp. 693–697. [Online]. Available: https://link.springer.com/10.1007/978-981-16-9012-921

[40] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette, "The NarrativeQA reading comprehension challenge," *Trans. Assoc. Comput. Linguistics*, vol. 6, pp. 317–328, Dec. 2018. [Online]. Available: https://aclanthology.org/Q18-1023

[41] K. Ganesan, "ROUGE 2.0: Updated and improved measures for evaluation of summarization tasks," 2018, *arXiv:1803.01937*.

[42] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2369–2380. [Online]. Available: http://aclweb.org/anthology/D18-1259

[43] S. Min, J. Michael, H. Hajishirzi, and L. Zettlemoyer, "AmbigQA: Answering ambiguous open-domain questions," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 5783–5797. [Online]. Available: https://www.aclweb.org/anthology/2020.emnlp-main.466

[44] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," 2020, *arXiv:2009.03300*.

[45] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, "Recitation-augmented language models," 2022, *arXiv:2210.01296*.

[46] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 453–466, Nov. 2019. [Online]. Available: https://direct.mit.edu/tacl/article/43518

[47] M. Joshi, E. Choi, D. Weld, and L. Zettlemoyer, "TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 1601–1611. [Online]. Available: http://aclweb.org/anthology/P17-1147

[48] B. Deng, W. Wang, F. Zhu, Q. Wang, and F. Feng, "CrAM: Credibility-aware attention modification in LLMs for combating misinformation in RAG," 2024, *arXiv:2406.11497*.

[49] Y. Chang, M. Narang, H. Suzuki, G. Cao, J. Gao, and Y. Bisk, "WebQA: Multihop and multimodal QA," 2021, *arXiv:2109.00590*.

[50] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2016, pp. 2383–2392. [Online]. Available: http://aclweb.org/anthology/D16-1264

[51] M. Glass, G. Rossiello, M. F. M. Chowdhury, A. Naik, P. Cai, and A. Gliozzo, "Re2G: Retrieve, rerank, generate," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, Jul. 2022, pp. 2701–2715. [Online]. Available: https://aclanthology.org/2022.naacl-main.194/

[52] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods Natural Lang. Process. 9th Int. Joint Conf. Natural Lang. Process. (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[53] I. Montani, M. Honnibal, A. Boyd, S. V. Landeghem, and H. Peters. (Oct. 2023). *Explosion/spaCy: V3.7.2: Fixes for APIs and Requirements*. [Online]. Available: https://zenodo.org/doi/10.5281/zenodo.1212303

[54] L. Tunstall, E. Beeching, N. Lambert, N. Rajani, K. Rasul, Y. Belkada, S. Huang, L. von Werra, C. Fourrier, N. Habib, N. Sarrazin, O. Sanseviero, A. M. Rush, and T. Wolf, "Zephyr: Direct distillation of LM alignment," 2023, *arXiv:2310.16944*.

[55] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," in *Proc. 8th Int. Conf. Learn. Represent.*, Apr. 2019, pp. 1–41.
[56] E. Collini, P. Nesi, and G. Pantaleo, "Deep learning for short-term prediction of available bikes on bike-sharing stations," *IEEE Access*, vol. 9, pp. 124337–124347, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/9530580/
[57] E. Collini, L. A. I. Palesi, P. Nesi, G. Pantaleo, and W. Zhao, "Flexible thermal camera solution for smart city people detection and counting," *Multimedia Tools Appl.*, vol. 83, no. 7, pp. 20457–20485, Aug. 2023. [Online]. Available: https://link.springer.com/10.1007/s11042-023-16374-x
[58] S. Bilotta, E. Collini, P. Nesi, and G. Pantaleo, "Short-term prediction of city traffic flow via convolutional deep learning," *IEEE Access*, vol. 10, pp. 113086–113099, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9930774/
[59] E. Collini, P. Nesi, and G. Pantaleo, "Reputation assessment and visitor arrival forecasts for data driven tourism attractions assessment," *Online Social Netw. Media*, vols. 37–38, Sep. 2023, Art. no. 100274. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2468696423000332
[60] E. Collini, L. A. I. Palesi, P. Nesi, G. Pantaleo, N. Nocentini, and A. Rosi, "Predicting and understanding landslide events with explainable AI," *IEEE Access*, vol. 10, pp. 31175–31189, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9732490/
[61] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," 2021, *arXiv:2112.09118*.
[62] C. Li, Z. Liu, S. Xiao, and Y. Shao, "Making large language models a better foundation for dense retrieval," 2023, *arXiv:2312.15503*.
[63] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2024, *arXiv:2402.03216*.
[64] F. Huth, M. Koch, M. Awad-Mohammed, D. Weiskopf, and K. Kurzhals, "Eye tracking on text reading with visual enhancements," in *Proc. Symp. Eye Tracking Res. Appl.*, Jun. 2024, pp. 1–7, doi: 10.1145/3649902.3653521.
[65] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022.

ENRICO COLLINI is currently pursuing the Ph.D. degree with the Department of Information Engineering, University of Florence. His research interests include deep learning, mobility, and data models.

FELIX INDRA KURNIADI received the bachelor's degree from Tarumanagara University, Indonesia, and the master's degree from Universitas Indonesia. He is currently pursuing the dual Ph.D. degree with the National Ph.D. in AI and Society, University of Pisa and the University of Florence, Italy. His research interests include statistical machine learning, deep learning, natural language processing, and explainability AI.

PAOLO NESI (Life Member, IEEE) is currently a Full Professor with the DINFO Department, University of Florence, and the Chief of the DISIT Laboratory. He is and has been the coordinator of several research and development multipartner international research and development projects. He has published more than 400 papers on international journals and conferences. His research interests include machine learning, artificial intelligence and explainable artificial intelligence (AI/XAI) massive parallel and distributed systems, physical models, the IoT, mobility, big data analytic, semantic computing, formal model, machine learning, and data privacy. He has been the chair of a number of international conferences.

GIANNI PANTALEO is currently an aggregated Professor with the DINFO Department, University of Florence. He has been a coordinator of several WPs in international research and development projects. His research interests include deep learning, natural language processing (NLP), large language models, knowledge engineering, the IoT, visual analytics, and semantic computing.